Exercise Sheet 2

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 13.4.2026 Deadline: 20.4.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 2.1 (*Introductory C code example of a serial* `axpy` *operation*).

Denote by $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ two vectors of dimension $n \in \mathbb{N}$ and let $\alpha \in \mathbb{R}$ be a given scalar. Write a serial C program to compute the `axpy` operation

$$\mathbf{y} \leftarrow \alpha \mathbf{x} + \mathbf{y}.$$

The dimension $n \in \mathbb{N}$ shall be passed as a command line argument. The implementation should use double-precision floating-point numbers and initialize both vectors with random values. To analyze performance, compile the programm without any optimization (using the `-O0` flag) and run it for suitable dimensions, for example, $n \in \{1024, 2048, 4096, \dots\}$. Measure only the runtime of the computational kernel excluding memory allocation and initialization for a repeated number of executions using a high-resolution timer. Visualize the results and compare them to a naive theoretical model only counting mathematical operations.

Exercise 2.2 (*Empirical Performance Analysis of naive Matrix-Vector Products*).

For dimension $n, m \in \mathbb{N}$ denote by $\mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^m$ vectors and let $A \in \mathbb{R}^{m \times n}$ be a given matrix. Write a serial C program to compute the matrix-vector product

$$\mathbf{y} \leftarrow A\mathbf{x}$$

The dimensions $n, m \in \mathbb{N}$ shall be passed as a command line argument. The implementation should use double-precision floating-point numbers and initialize the matrix A and the vector $\vec{x}$ with random values. To analyze performance, compile the programm without any optimization (using the `-O0` flag) and run it for suitable dimension, for example, $n \in \{256, 512, 1024, \dots\}$. Measure only the runtime of the computational kernel excluding memory allocation and initialization for a repeated number of executions using a high-resolution timer. Visualize the results and compare them to a naive theoretical model only counting mathematical operations.

Exercise 2.3 (*Empirical Performance Analysis of naive Matrix-Matrix Products*).

Denote by $A \in \mathbb{R}^{n \times m}, \mathbf{B} \in \mathbb{R}^{m \times l}$ two given matrices with dimensions $n, m, l \in \mathbb{N}$ and let $C \in \mathbb{R}^{n \times l}$. Write a serial C program to compute the matrix-matrix product

$$C \leftarrow AB.$$

The dimensions $n, m, l \in \mathbb{N}$ shall be passed as a command line argument. The implementation should use double-precision floating-point numbers and initialize both matrices with random values. To analyze performance, compile the programm without any optimization (using the `-O0` flag) and run it for suitable dimension, for example, $n = m = l \in \{64, 128, 256, \dots\}$. Measure only the runtime of the computational kernel excluding memory allocation and initialization for a repeated number of executions using a high-resolution timer. Visualize the results and compare them to a naive theoretical model only counting mathematical operations.

 © Advanced Computing 2026